

普及组初赛模拟题（二）

一、单项选择题（共15题，每题2分，共计30分；每题有且仅有一个正确选项）

1. 每个不同的二进制数可以表示一位学生，现要用二进制数来表示 1200 位学生，至少需要二进制数的位数是：

- A. 11
- B. 10
- C. 12
- D. 13

2. 若有如下程序段，其中 s 、 x 、 y 、 z 均已定义为 `int` 类型变量，且 x 、 z 均已赋值（ z 大于 0）。

```
s = x;
for (y = z; y >= 1; y--) {
    s = s + 1;
}
```

则与上述程序段修改 s 值的功能等价的赋值语句是（ ）。

- A. $s = y + z;$
- B. $s = x + y;$
- C. $s = s + z;$
- D. $s = x + z;$

3. 已知有序表 (13, 18, 24, 35, 47, 50, 62, 83, 90, 115, 134)，当折半查找值为 90 的元素时，查找成功的比较次数为（ ）。

- A. 4
- B. 5
- C. 3
- D. 2

4. 已知大写字母 A 的 ASCII 编码为 65（十进制），则大写字母 J 的十进制 ASCII 编码为：

- A. 71
- B. 72
- C. 73
- D. 74

5. 设 $x = \text{true}$ ， $y = \text{true}$ ， $z = \text{false}$ ，以下逻辑表达式值为真的是（ ）。

- A. $(x \vee y) \wedge z$
- B. $x \wedge (z \vee y) \wedge z$
- C. $(x \vee y) \wedge (x \vee z)$
- D. $(y \vee z) \wedge x \wedge z$

6. 一个二叉搜索树前序遍历的结果为 7、2、1、5、13、9，这棵树的根的左子树有多少个节点？
()。

- A. 2
- B. 5
- C. 4
- D. 3

7. 某算法计算时间表示为递推关系式： $T(N) = N + T(\frac{N}{2})$ ，则该算法时间复杂度为()。

- A. $O(N^2)$
- B. $O(N \log N)$
- C. $O(N)$
- D. $O(1)$

8. 共 9 个互不相同的数，它们的最大公约数是 2021 的一个大于 1 的因子(6 有 2、3、6 这三个大于 1 的因子，因子可以包含自身)，且这 9 个数的和小于等于 2021，则这 9 个数的和是多少?()

- A. 1849
- B. 1935
- C. 2021
- D. 1927

9. 一个 n 个顶点的强连通图最少有几条边()

- A. n
- B. $n + 1$
- C. $n - 1$
- D. $n \times (n - 1)$

10. 蒜头君计划展出 10 幅不同的画，其中 1 幅水彩画、4 幅油画、5 幅国画，排成一行陈列，要求同一品种的画必须连在一起，并且水彩画不放在两端，那么不同的陈列方式有()种。

- A. 5760
- B. 2880
- C. 17280
- D. 8640

11. 在一个图中，所有顶点的度数之和等于所有边数的()倍。

- A. 2
- B. 4
- C. 1
- D. $\frac{1}{2}$

12. 6 个人分乘两辆不同的汽车，每辆车最多坐 4 人，则不同的乘车方法数为()。

- A. 40
- B. 50
- C. 70
- D. 60

13. 设 G 是有 6 个结点的完全图，要得到一颗生成树，需要从 G 中删去 () 条边。

- A. 6
- B. 9
- C. 10
- D. 15

14. 8 颗子弹，编号为 1、2、3、4、5、6、7、8，从编号 1 开始按序嵌入弹夹，以下有哪个不是正常的打出子弹的次序 ()

- A. 87654321
- B. 32154876
- C. 32164587
- D. 12345678

15. 某公司派赵钱孙李周五人出国学习，选派条件是：

- a. 若赵去，钱也去；
- b. 李、周两人必有一人去
- c. 如周去，则赵、钱也同去；
- d. 孙、李二人同去或同不去

如何选他们出国？ ()

- A. 孙赵周去
- B. 李周孙去
- C. 赵钱周去
- D. 钱孙去

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

注意：判断题正确填 T，错误填 F。

阅读下面程序，完成第 16 ~ 21 题。

```

1  #include <iostream>
2
3  #include <cstring>
4
5  using namespace std;
6  int main() {
7      int t[256];
8      char s[10];
9      int i;
10     scanf("%s", s);
11     for (i = 0; i < 256; i++) t[i] = 0;
12     for (i = 0; i < strlen(s); i++) t[s[i]]++;
13     for (i = 0; i < strlen(s); i++)
14         if (t[s[i]] == 1) {
15             cout << s[i] << endl;
16             return 0;
17         }
18     cout << "no" << endl;
19     return 0;
20 }

```

16. 程序会将读入的字符串中出现次数等于 1 的字符依次输出。[]
17. 将程序中的 `for (i = 0; i < 256; i++) t[i] = 0;` 删掉可能会影响程序的正确性。[]
18. 当程序读入字符串的内容是 "xyzxyw" 时，程序的输出结果是 w。[]
19. 对于一组输入数据 "abc?ac"，如果希望程序的输出结果为 b，则 ? 处应替换为 b。[]
20. 若输入的字符串中 a 到 g 这 7 种字符均至少出现一次，如果希望输出结果为 no，则输入的字符串长度至少为14。[]
21. 这个程序最多正确输入并处理长度为()的字符串 s。
 - A. 256
 - B. 10
 - C. 9
 - D. 11

阅读下面程序，完成第 22 ~ 27 题。

```

1  #include<iostream>
2  #include<cstdio>
3
4  using namespace std;
5
6  const int MAXN = 100;
7
8  int arr[MAXN];
9
10 bool BinarySearch(int n, int target) {
11     int left = 0;
12     int right = n- 1;
13     while (left <= right) {
14         int middle = (left + right) / 2;
15         if (arr[middle] < target) {
16             left = middle + 1;
17         } else if (target < arr[middle]) {
18             right = middle- 1;
19         } else {
20             return true;
21         }
22     }
23     return false;
24 }
25
26 int main() {
27     int n, m;
28     cin >> n;
29     for (int i = 0; i < n; i++)
30         cin >> arr[i];
31     sort(arr, arr + n); //(1)将arr数组中的元素从小到大排序
32     cin >> m;
33     for (int i = 0; i < m; i++) {
34         int target;
35         cin >> target;
36         if (BinarySearch(n, target)) {
37             cout << "YES" << endl;
38         } else {
39             cout << "NO" << endl;
40         }
41     }
42     return 0;
43 }

```

22. 该算法的时间复杂度是 $O(m \log n)$ 。(不考虑(1)的排序) ()。

23. 将 `while (left <= right)` 改成 `while(left < right)`，其他地方不做改动，对程序最终的输出结果没有影响。()

24. 将 `sort(arr, arr + n);` 改成把 `arr` 数组中的元素从大到小排序的代码，其他地方不做改动，对程序最终的输出结果没有影响。（ ）
25. 将 `int middle = (left + right) / 2;` 改成下列选项中的哪一项，对于程序的运行没有影响。（ ）
- A. `int middle = (left + right) >> 1;`
 - B. `int middle = (left + right) << 1;`
 - C. `int middle = (left + right) > 1;`
 - D. `int middle = (left + right) < 1;`
26. 若给定 n 和 `arr` 数组，在最好情况下，函数 *BinarySearch* 中的 `while` 循环需要被执行（ ）次。
- A. n
 - B. m
 - C. $m \times \lceil \log(n) \rceil$
 - D. $m \times \lfloor \log(n) \rfloor$
27. (4分) 若输入如下数据：

```
5
1 5 2 4 3
3
2 5 6
```

则输出结果为(用空格表示换行)：（ ）。

- A. YES YES YES
- B. YES NO YES
- C. YES YES NO
- D. NO NO NO

阅读下面程序，完成第 28 ~ 33 题。

```

1  #include <iostream>
2
3  #include <algorithm>
4
5  #include <stdio.h>
6
7  using namespace std;
8  int w[35000], d[35000], dp[35000];
9  int main() {
10     int n, m;
11     scanf("%d%d", &n, &m);
12     for (int i = 1; i <= n; i++)
13         scanf("%d", &d[i]);
14     for (int i = 1; i <= n; i++) {
15         for (int j = m; j >= w[i]; j--) {
16             dp[j] = max(dp[j], dp[j - w[i]] + d[i]);
17         }
18     }
19     printf("%d\n", dp[m]);
20     return 0;
21 }

```

28. 上述代码中，双重循环里循环变量 j 的枚举顺序改为从 $w[i]$ 到 m ，输出结果一定不变。[]
29. 上述代码中，双重循环中变量 i 的枚举顺序改为从 n 到 1 ，输出结果一定不变。[]
30. 若输入数据中， $1 \leq n \leq 30000, 1 \leq m \leq 30000, 1 \leq w[i] \leq 30000, 1 \leq d[i] \leq 30000$ ，则所求答案一定没有溢出。[]
31. 当输入为：

```

4 6
1 4
2 6
3 12
2 7

```

输出为 ()

- A. 17
 - B. 28
 - C. 29
 - D. 23
32. (4分) 若输入数据中， $1 \leq n \leq 30000, 1 \leq m \leq 30000, 1 \leq w[i] \leq 30000$ ，下列给出的 $d[i]$ 的范围，哪个选项所求答案有可能会超出 INT 的范围 ()
- A. $1 \leq d[i] \leq 100$
 - B. $1 \leq d[i] \leq 1000$
 - C. $1 \leq d[i] \leq 10000$
 - D. $1 \leq d[i] \leq 100000$

33. 上述代码的时间复杂度为 ()

- A. $O(n)$
- B. $O(n^2m)$
- C. $O(nm)$
- D. $O(nm^2)$

三、完善程序（单选题，每小题3分，共计30分）

阅读下面题目，完成第 34 ~ 38 题。

题目描述

在二维平面内有若干个点，每个点的视野价值定义为其左下方的点的数目，即 $val_i = \sum_j [x_j < x_i] \text{ and } [y_j < y_i]$ ，求出每个点的视野价值并输出最具有视野价值的点的编号，如果有多个这样的点，输出其中最大的编号。

输入说明

第一行一个正整数 n ，表示二维平面中点的个数。

接下来 n 行，每行两个数，表示编号为 i 的点的坐标。

输出说明

第一行 n 个正整数，表示编号为 i 的点的视野价值。

第二行一个正整数，表示视野价值最高的点的编号，如果有多个视野价值相同的点，则输出编号最大的那个。

样例输入

```
5
2 3
3 2
1 1
3 5
5 3
```

样例输出

```
1 1 0 2 2
5
```


请补全下面的代码。

```
1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  const int N = 100;
6  int x[N], y[N], f[N];
7  int n, max_f, ans;
8
9  int main() {
10     cin >> n;
11     for (int i = 1; i <= n; i++) cin >> x[i] >> y[i];
12     max_f = 0;
13     for (int i = 1; i <= n; i++) {
14         f[i] = ①;
15         for (int j = 1; j <= n; j++) {
16             if (x[j] < x[i] && ②)
17                 ③;
18         }
19         if (④) {
20             max_f = f[i];
21             ⑤;
22         }
23     }
24     for (int i = 1; i <= n; i++) cout << f[i] << " ";
25     cout << endl;
26     cout << ans << endl;
27     return 0;
28 }
```

34. ①处应填()

- A. 0
- B. 1
- C. i
- D. INT_MAX

35. ②处应填()

- A. $y[j] \leq y[i]$
- B. $y[j] < y[i]$
- C. $y[j] > y[i]$
- D. $y[j] \geq y[i]$

36. ③处应填()

- A. $ans = i$
- B. $f[i]++$
- C. $f[i]--$

- D. ans++

37. ④处应填()

- A. (f[0]>max_f)
- B. (f[i]>=max_f)
- C. (f[i]<max_f)
- D. (f[i]<=max_f)

38. ⑤处应填()

- A. ans++
- B. ans+=max_f
- C. ans=i
- D. ans= max_f

阅读下面题目，完成第 39 ~ 43 题。

题目描述

蒜头君的幼儿园有 n 个小朋友，在一次升旗仪式中，共有 n 个可以上场的小朋友，现在共需要 m 个小朋友来组成仪仗队，请你列举出所有可能的仪仗队，并按照字典序进行输出。

输入说明

输入共一行，两个正整数 n, m ，分别表示能上场的小朋友数目和仪仗队的人数。

输出说明

输出有若干行，每一行有 m 个数，表示一种可能的仪仗队组成形式，并按照字典序输出。

样例输入

3 2

样例输出

1 2
1 3
2 1
2 3
3 1
3 2

请补全下面的代码。

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  const int N = 25;
6
7  int n, m;
8  bool flag;
9  bool used[N];
10 int data[N];
11
12 int main() {
13     cin >> n >> m;
14     memset(used, false, sizeof(used));
15     for (int i = 1; i <= m; ++i) {
16         data[i] = i;
17         used[i] = true;
18     }
19     flag = true;
20     while (flag) {
21         for (int i = 1; i <= m - 1; i++) cout << data[i] << " ";
22         cout << data[m] << endl;
23         flag = ①;
24         for (int i = m; i >= 1; i--) {
25             ②;
26             for (int j = data[i] + 1; j <= n; j++) {
27                 if (!used[j]) {
28                     used[j] = true;
29                     data[i] = ③;
30                     flag = true;
31                     break;
32                 }
33             }
34             if (flag) {
35                 for (int k = i + 1; k <= m; k++) {
36                     for (int j = 1; j <= ④; j++) {
37                         if (!used[i]) {
38                             data[k] = j;
39                             used[j] = true;
40                             break;
41                         }
42                     }
43                 }
44                 ⑤;
45             }
46         }
47     }
48     return 0;
49 }

```

39. ① 处应填()

- A. false
- B. true
- C. 1
- D. -1

40. ② 处应填()

- A. used[i]= true
- B. data[i]=i
- C. used[data[i]]= true
- D. used[data[i]]= false

41. ③ 处应填()

- A. j
- B. i
- C. true
- D. false

42. ④ 处应填()

- A. n
- B. m
- C. i
- D. j

43. ⑤ 处应填()

- A. return 0
- B. exit
- C. continue
- D. break